

Technical Design Document (TDD)

Product Name

Restaurant Knowledge OS

Document Purpose

This document defines the technical architecture, system components, data model, APIs, execution flow, deployment strategy, security model, and implementation plan for Restaurant Knowledge OS.

The goal is to build a multilingual, self-hosted, restaurant-specific AI receptionist platform that allows owners to teach their business knowledge through natural conversation and allows customers to interact with that knowledge in real time through text or voice.

1. System Overview

Restaurant Knowledge OS is a multi-tenant AI platform with the following major capabilities:

1. Restaurant owner onboarding
2. Conversational knowledge capture
3. Structured information extraction
4. Knowledge validation and versioning
5. Semantic and exact retrieval
6. Customer-facing chat and voice interaction
7. Tenant isolation and secure data storage
8. Multilingual support for Hindi, Gujarati, and English first
9. Local-first deployment with optional later cloud scaling

The platform does not fine-tune the base LLM in the MVP. Instead, it uses Retrieval-Augmented Generation (RAG) over restaurant-specific knowledge.

2. Technical Goals

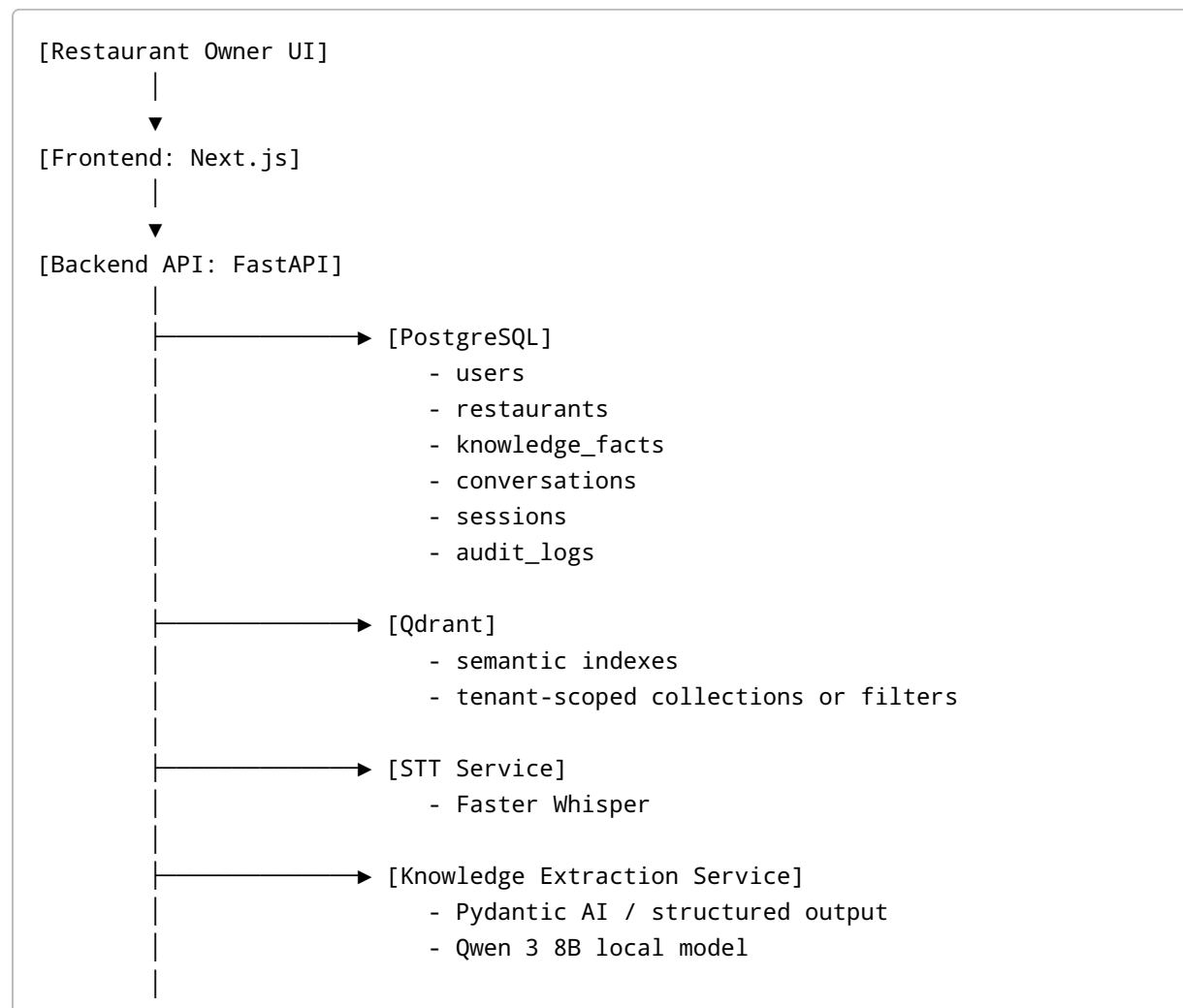
- Build a modular but simple architecture that can run on the founder's hardware.
 - Keep the system fully functional without mandatory proprietary AI APIs.
 - Support one restaurant, then scale to many restaurants without data leakage.
 - Use structured knowledge rather than raw chat history as the source of truth.
 - Maintain low latency for both text and browser-based voice experiences.
 - Keep the codebase easy to debug and replaceable component by component.
-

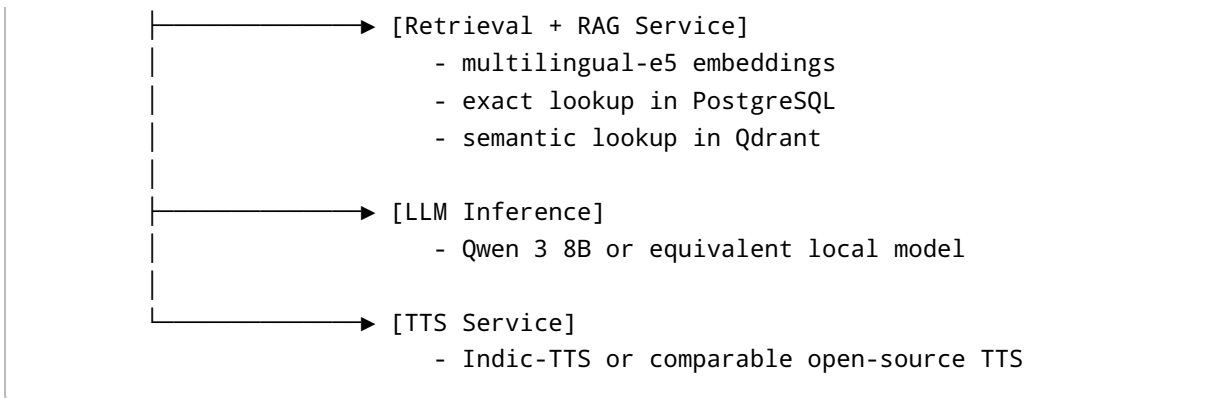
3. Non-Goals for MVP

The following are intentionally excluded from the initial implementation:

- real phone call routing
 - WhatsApp integration
 - paid billing/subscription logic
 - advanced analytics
 - model fine-tuning
 - voice cloning
 - CRM integration
 - appointment payments
 - warehouse/inventory management
 - Kubernetes from day one
-

4. High-Level Architecture





The MVP should be implemented as a **modular monolith** first, not microservices everywhere. Each service boundary should exist in code, but deployment can be a single backend process with separate modules and optionally separate containers.

5. Recommended Implementation Philosophy

Phase 1: Modular Monolith

Keep all backend logic under one FastAPI application with clean modules:

- auth
- restaurants
- training
- extraction
- retrieval
- chat
- voice
- knowledge
- tenant isolation

Phase 2: Split Only If Needed

If latency, maintenance, or deployment complexity grows, split the following into separate services:

- STT
- TTS
- LLM inference
- retrieval
- agent orchestration

This avoids early overengineering.

6. Core Technology Decisions

Frontend

Next.js

- fast iteration
- good for dashboards and interactive UI
- can host browser-based voice interactions

Backend

FastAPI

- strong Python ecosystem for AI
- async capable
- easy API design
- good integration with Pydantic AI, Qdrant, and PyTorch tooling

SQL Database

PostgreSQL

- source of truth for structured restaurant facts
- supports multi-tenancy well
- good for audit logs and versioning

Vector Database

Qdrant

- self-hostable
- fast
- good tenant isolation strategies
- easy API access

STT

Faster Whisper

- multilingual
- local inference
- good performance on RTX 4060

Embeddings

multilingual-e5

- cross-language retrieval
- owner can teach in Gujarati and customer can ask in Hindi

LLM

Qwen 3 8B or equivalent local multilingual instruct model

- practical on RTX 4060 with quantization
- strong multilingual ability
- suitable for structured extraction and response generation

TTS

Indic-TTS or comparable open-source multilingual TTS

- Hindi / Gujarati / English support
- local inference

Structured Output / Validation

Pydantic AI

- strongly typed outputs
 - easy schema enforcement
 - good for extraction tasks
-

7. Core Domain Model

7.1 Tenant Model

A tenant is a restaurant. Every restaurant has:

- one owner account or multiple authorized users later
- one isolated knowledge space
- one conversation history space
- one settings configuration
- one semantic namespace

7.2 Knowledge Model

The system stores not raw conversation as truth, but structured facts. Examples:

- business hours
- menu items
- prices
- delivery rules
- offers
- holiday closures
- contact numbers
- payment methods
- reservation policy
- special instructions

7.3 Conversation Model

Conversation records are stored for audit, improvement, and debugging. They are not the source of truth unless promoted into knowledge.

8. Data Architecture

8.1 PostgreSQL Tables

users

- id
- name
- email
- password_hash
- role
- created_at
- updated_at

restaurants

- id
- owner_id
- name
- cuisine_type
- language_preferences
- timezone
- location
- phone_number
- status
- created_at

- updated_at

restaurant_members

- id
- restaurant_id
- user_id
- role
- created_at

knowledge_facts

- id
- restaurant_id
- category
- fact_key
- fact_value_json
- source_text
- source_language
- confidence_score
- status
- version
- created_at
- updated_at
- deleted_at

knowledge_versions

- id
- fact_id
- restaurant_id
- previous_value_json
- new_value_json
- changed_by
- change_reason
- created_at

conversations

- id
- restaurant_id
- session_id
- speaker_type
- input_language
- input_text
- output_text
- created_at

sessions

- id
- restaurant_id
- customer_identifier
- channel
- state
- created_at
- updated_at

audit_logs

- id
- restaurant_id
- actor_id
- action
- entity_type
- entity_id
- metadata_json
- created_at

extraction_jobs

- id
- restaurant_id
- source_text
- source_language
- extracted_json
- validation_status
- created_at
- updated_at

8.2 Qdrant Strategy

Use one of two approaches:

Option A: Collection per restaurant

Best for MVP.

- strong isolation
- easy to reason about
- simple debugging
- acceptable at low scale

Option B: Shared collection with tenant_id filter

Best for larger scale.

- lower operational overhead
- more efficient for many tenants
- requires stricter query filtering

Recommendation

- MVP: collection per restaurant
 - scale stage: shared collection + tenant filter
-

9. Knowledge Schema Design

Each extracted fact should map to a strongly structured object.

Example: Business Hours

```
{
  "type": "business_hours",
  "day": "Sunday",
  "status": "closed"
}
```

Example: Menu Item

```
{
  "type": "menu_item",
  "name": "Paneer Pizza",
  "price": 250,
  "currency": "INR",
  "availability": "available"
}
```

Example: Delivery Policy

```
{
  "type": "delivery_policy",
  "radius_km": 5,
  "delivery_fee": 0,
}
```

```
  "notes": "Free delivery within 5 km"
}
```

Example: Reservation Rule

```
{
  "type": "reservation_policy",
  "advance_notice_hours": 2,
  "party_size_limit": 20
}
```

Example: Offer

```
{
  "type": "offer",
  "name": "Birthday Discount",
  "discount_percent": 20,
  "condition": "Birthday party booking"
}
```

10. Knowledge Lifecycle

10.1 Ingestion

Restaurant owner provides data by text or voice.

10.2 Transcription

Voice is converted to text using Faster Whisper.

10.3 Extraction

LLM converts conversation into structured facts.

10.4 Validation

System checks for:

- schema validity
- duplicates
- conflicts
- missing values
- contradictory changes

10.5 Approval

Owner can approve, edit, or reject extracted facts.

10.6 Storage

Approved facts are written to PostgreSQL and indexed in Qdrant.

10.7 Retrieval

Customer queries use SQL first for exact facts and Qdrant second for semantic matches.

10.8 Update

When a fact changes, the old fact is versioned or marked inactive, and the new fact replaces it.

11. Retrieval Strategy

The retrieval system should be hybrid.

11.1 Exact Lookup

Use PostgreSQL when the question maps to a precise fact. Examples:

- opening hours
- phone number
- price of a specific menu item
- holiday closure

11.2 Semantic Retrieval

Use Qdrant when the question is fuzzy or phrased differently. Examples:

- "Are you open tomorrow?"
- "Do you deliver nearby?"
- "Is there any birthday offer?"

11.3 Retrieval Flow

1. Detect tenant
2. Classify query intent
3. Attempt exact SQL lookup
4. Fall back to semantic search in Qdrant
5. Build prompt with only retrieved context
6. Generate answer
7. Return text or voice output

12. LLM Strategy

The LLM has two major jobs:

12.1 Extraction Job

Convert owner speech into structured facts. This requires constrained output.

12.2 Answer Generation Job

Generate a customer-facing response using only retrieved context.

LLM Rules

- do not hallucinate outside supplied context
- answer only from restaurant knowledge when possible
- if uncertain, ask for clarification or say the system does not know
- preserve customer language when possible

Recommended Prompting Pattern

Use a strict system prompt with:

- role
- tenant context
- retrieved facts
- language constraint
- refusal policy for missing facts

13. Structured Output / Guardrails

The platform must not accept arbitrary LLM text as truth.

Required Validation Layers

- JSON schema validation
- Pydantic model parsing
- conflict detection
- confidence thresholding
- human confirmation for uncertain facts

Example Validation

If the owner says:

- “Sunday closed”
- later “Sunday open”

The system should not blindly overwrite. It should ask:

You previously marked Sunday as closed. Do you want to update it?

14. Multilingual Strategy

The initial multilingual targets are:

- Gujarati
- Hindi
- English

14.1 Input Language Handling

- detect language automatically
- store source language on every knowledge entry
- preserve original phrasing where useful

14.2 Cross-Language Retrieval

Use multilingual embeddings so that:

- owner teaches in Gujarati
- customer asks in Hindi
- system still retrieves correct knowledge

14.3 Output Language Handling

- respond in the customer’s language when possible
- if the customer uses mixed language, respond in the dominant language

14.4 TTS Language Handling

- choose TTS voice based on customer language
 - use separate voice profiles for Hindi, Gujarati, English
-

15. Voice System Design

15.1 Owner Voice Training

Flow:

- owner speaks
- STT transcribes
- extractor identifies facts
- facts are shown in review UI
- owner approves or edits

15.2 Customer Voice Chat

Flow:

- customer speaks
- STT transcribes
- retrieval engine fetches relevant facts
- LLM generates answer
- TTS speaks answer back

15.3 Latency Strategy

To keep latency low:

- use smaller local models first
 - keep voice in browser for MVP
 - avoid real telephony early
 - stream partial output if possible later
-

16. API Design

16.1 Auth APIs

- POST /auth/signup
- POST /auth/login
- POST /auth/logout
- GET /auth/me

16.2 Restaurant APIs

- POST /restaurants
- GET /restaurants/:id
- PATCH /restaurants/:id

- GET /restaurants

16.3 Knowledge APIs

- POST /restaurants/:id/knowledge/extract
- POST /restaurants/:id/knowledge/approve
- PATCH /restaurants/:id/knowledge/:fact_id
- DELETE /restaurants/:id/knowledge/:fact_id
- GET /restaurants/:id/knowledge
- GET /restaurants/:id/knowledge/history

16.4 Chat APIs

- POST /restaurants/:id/chat
- POST /restaurants/:id/voice-chat
- GET /restaurants/:id/conversations

16.5 Voice APIs

- POST /stt/transcribe
- POST /tts/speak

16.6 Admin APIs

- GET /admin/restaurants
 - GET /admin/metrics
 - GET /admin/audit-logs
-

17. Frontend Architecture

Main Pages

- landing page
- signup/login page
- restaurant dashboard
- training page
- knowledge review page
- customer test page
- settings page
- conversation history page

Training Page Behavior

The training page should have two modes: 1. guided input mode 2. free conversation mode

The guided mode should be default for better extraction quality.

Customer Test Page Behavior

The customer test page should include:

- text input
 - mic button
 - response panel
 - source facts panel
 - language selector
-

18. Backend Module Breakdown

```
backend/
├── app/
│   ├── auth/
│   ├── restaurants/
│   ├── knowledge/
│   ├── extraction/
│   ├── retrieval/
│   ├── chat/
│   ├── voice/
│   ├── stt/
│   ├── tts/
│   ├── tenants/
│   ├── sessions/
│   ├── audit/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── utils/
│   └── config/
```

Responsibilities

auth

Authentication and session management.

restaurants

Restaurant CRUD and tenant setup.

knowledge

Knowledge CRUD, versioning, review, approval, and deletion.

extraction

LLM-based extraction and JSON parsing.

retrieval

Exact lookup and semantic retrieval orchestration.

chat

Customer-facing query handling.

voice

Audio input/output orchestration.

stt

Speech-to-text integration.

tts

Text-to-speech integration.

tenants

Tenant resolution and data isolation helpers.

sessions

Session tracking and conversation state.

audit

Logs and administrative history.

19. Deployment Architecture

19.1 MVP Deployment

Single machine deployment using Docker Compose.

Containers:

- frontend
- backend
- postgres
- qdrant
- optional model server

19.2 Production Deployment Later

- separate GPU inference node
- backend API node
- vector DB node
- SQL DB node
- monitoring node
- optional load balancer

19.3 Environment Targets

- local laptop for development
 - Linux server for staging and production
 - optional cloud GPU later if revenue justifies it
-

20. Model Serving Strategy

Option A: Local Python Loading

Simple for MVP.

- load model in FastAPI service
- easiest to build
- good for prototype and initial demo

Option B: Separate Inference Server

Better for scale.

- better isolation
- easier to restart
- allows horizontal expansion

Recommendation

Start with Option A. Move to Option B only when needed.

21. Security Design

21.1 Tenant Isolation

Every database query must include `restaurant_id`. Every retrieval call must be tenant-scoped. Every session must be tenant-bound.

21.2 Authentication

- JWT or session-based auth
- role checks for owner/admin
- secure password storage

21.3 Input Sanitization

- reject malformed JSON
- validate all structured extraction results
- sanitize prompt inputs
- avoid prompt injection via user text

21.4 Logging Policy

- do not store sensitive raw audio unnecessarily
- log metadata where possible
- redact personal data when needed

21.5 Data Protection

- HTTPS for all APIs
 - encrypted backups
 - controlled admin access
-

22. Error Handling Strategy

Extraction Failures

If the LLM cannot extract cleanly, store the conversation as pending review.

Retrieval Failures

If no knowledge is found, respond with a safe fallback:

- "I could not find that information."
- or ask the owner to update the knowledge base.

STT Failures

If transcription confidence is low, ask the user to repeat.

TTS Failures

Fall back to text response.

Tenant Resolution Failures

Never answer without tenant identification.

23. Testing Strategy

Unit Tests

- extraction schema parsing
- tenant filtering
- retrieval logic
- update/versioning logic
- auth logic

Integration Tests

- owner training flow
- customer query flow
- cross-language retrieval
- knowledge update flow

Security Tests

- cross-tenant data leakage attempts
- invalid input injection
- unauthorized update attempts

Voice Tests

- STT accuracy on Hindi, Gujarati, English samples
- TTS output validation
- end-to-end audio roundtrip

Load Tests

- multiple restaurant queries
 - concurrent session handling
 - retrieval latency under load
-

24. Observability

The system should log:

- request duration
- extraction confidence
- retrieval source used
- tenant ID
- language detected
- response duration
- error traces

Future Monitoring Tools

- Prometheus
- Grafana
- Loki
- Sentry

For MVP, keep monitoring light.

25. Versioning Strategy

Knowledge changes must be versioned.

Example Update Flow

1. old fact exists
2. owner updates it
3. system stores new version
4. old fact is marked inactive or archived
5. new embeddings are generated
6. retrieval uses active version only

This prevents data corruption and makes rollbacks possible.

26. Recommended Development Order

Week 1

- repo setup
- database schema
- auth
- restaurant onboarding
- knowledge schema definitions

Week 2

- extraction engine
- knowledge approval UI
- knowledge storage
- versioning

Week 3

- customer chat flow
- SQL retrieval
- Qdrant retrieval

Week 4

- multilingual support
- voice input/output in browser

Week 5

- tenant isolation hardening
 - testing
 - deployment packaging
-

27. Production Scaling Strategy

Stage 1: Single Restaurant

- single tenant
- single machine
- basic browser voice

Stage 2: 10 Restaurants

- same architecture
- per-tenant knowledge separation
- improved admin tools

Stage 3: 100 Restaurants

- optimize retrieval
- introduce caching
- separate model server if needed

Stage 4: 1000+ Restaurants

- shard vector data
- split inference services

- add load balancer
 - add monitoring
-

28. Final Technical Positioning

Restaurant Knowledge OS should be treated as a **private knowledge and conversation platform**, not as a model training platform.

The base model is shared. The restaurant knowledge is private. The retrieval layer is tenant-scoped. The user experience is conversational. The architecture is self-hostable and production-oriented.

This is the correct foundation for building a real SaaS product from the same codebase as the college project.

29. Immediate Next Technical Tasks

1. define DB schema in SQL
 2. define Pydantic models for all knowledge types
 3. define API contracts
 4. create modular backend skeleton
 5. implement restaurant onboarding
 6. implement knowledge extraction
 7. implement knowledge review/edit UI
 8. implement customer chat with exact + semantic retrieval
 9. add browser voice
 10. test tenant isolation
-

30. Technical Success Criteria

The TDD is successful if the architecture:

- can run locally
- supports Hindi/Gujarati/English
- keeps each restaurant isolated
- uses open-source components
- avoids fine-tuning dependency
- supports structured knowledge updates
- can scale beyond a demo without rewrite